

Chapter 9

Training and Adapting Vision-Language Models

Exam Study Guide, Worked Concepts, Sample Questions, and

Prepared from	uploaded course materials
Main source	Chapter_9_Training_and_Adapting_VLMs(1).ipynb
Exam source	Chapter_9_Summary_and_Sample_Questions.pdf
Practice source	Chapter_9_Flashcards.pdf

How to use it

Study adaptation as a ladder of interventions. Ask what changes: prompt, context, retrieved evidence, trainable parameters, preference signal, or reward optimization.

Study order: First read the concept sections, then solve the five sample exam questions, then test yourself with the flashcards.

Contents

- 1. Quick exam map
- 2. Common autoregressive VLM policy
- 3. Zero-shot and few-shot prompting
- 4. RAG as evidence injection
- 5. Supervised fine-tuning and masking
- 6. LoRA and QLoRA
- 7. DPO and RL-based alignment
- 8. Choosing the right intervention
- 9. Formula cheat sheet and symbol key
- 10. Code prompts
- 11. Exam-style questions
- 12. Flashcards

Most useful mental shortcut
Zero-shot, few-shot, and RAG mostly change context; SFT, LoRA, DPO, and RL change the policy or its trainable update path.

1. Quick exam map

This chapter explains how to adapt a pretrained VLM without confusing prompting, retrieval, fine-tuning, preference tuning, and reinforcement learning.

Base VLM policy

$p_{\theta}(y|\text{text,image})$

Prompting

change instruction/context

RAG

add retrieved evidence

SFT/LoRA

update behavior efficiently

DPO/RL

align with preferences or rewards Figure: the main method pipeline for this chapter.

Exam topic	What you must be able to say
Common VLM policy	$p_{\theta}(y x_{\text{text}},x_{\text{image}})$ autoregressively generates responses.
Zero/few-shot	No parameter update; behavior changes through context.
RAG	Inject retrieved evidence at inference time.

Exam topic	What you must be able to say
SFT	Train on labeled examples with next-token likelihood.
LoRA/QLoRA	Freeze base weights and learn low-rank updates.
DPO/RL	Use preferences or rewards to align outputs.

2. Zero-prior explanation: understand the

idea before the equations This section assumes you have not read the lecture yet. The goal is to build intuition first, then connect the intuition to the equations and exam language.

Adaptation ladder

Methods differ by what they change. Prompting changes instructions. Few-shot adds demonstrations. RAG adds external evidence. SFT changes weights using labels. DPO and RL align behavior using preferences or rewards.

Zero-shot and few-shot

No gradient update occurs. The model behavior changes because the Transformer conditions on the full context, including examples.

RAG

Retrieval-augmented generation is best when the model lacks knowledge. It conditions the answer on retrieved evidence rather than relying only on internal parameters.

Memory line

Before adapting a model, diagnose what kind of failure you are fixing.

3. Core concepts and worked explanations

SFT and masking

Supervised fine-tuning trains the model on labeled responses. In chat data, the loss usually applies only to assistant tokens, not user/system text.

LoRA and QLoRA

LoRA freezes the base weight and learns a low-rank update. QLoRA reduces memory further by quantizing base weights while training low-rank adapters.

DPO and RL

DPO compares preferred and rejected responses relative to a reference policy. RL treats token generation as a sequence of actions and optimizes expected reward.

4. Method explanation, worked example,

and application

Step	Method explanation
Start with prompting	Change instruction without changing parameters.
Add evidence	Use RAG when failure is missing or stale knowledge.
Fine-tune efficiently	Use SFT/LoRA for persistent task behavior.
Align behavior	Use DPO/RL when preferences or rewards matter.

Worked example

Example: if a VLM confuses two similar species, RAG can provide a field-guide passage. LoRA SFT can specialize on labeled examples. DPO can later prefer concise grounded answers over vague ones.

Applications and usage

Usage: these methods build domain VLMs, grounded assistants, medical or industrial visual systems, retrieval-augmented QA tools, and preference-aligned chat models.

5. Formula / definition cheat sheet

Read formulas as compact sentences. Do not memorize symbols blindly; use the symbol key table after the formula table to understand what each part represents.

Formula	Interpretation
$p\theta(y x,v) = \prod_t p\theta(y_t y_{<t}, x, v)$ Autoregressive VLM policy.	$p(y x,v) = \sum_d p(y x,v,d)p(d x,v)$ RAG evidence marginalization.
$L = -\sum_t m_t \log p\theta(y_t y_{<t}, x, v)$ Masked supervised fine-tuning.	$W' = W + BA$ LoRA low-rank update.
$LDPO = -\log \sigma(\beta[(\log\pi\theta(yw) - \log\pi\theta(y_l)) - (\log\pi_{ref}(yw) - \log\pi_{ref}(y_l))])$ Preference alignment.	$A_i = r_i - \text{mean}_{\{j \neq i\}} r_j$ RLOO relative reward baseline.

6. Equation symbol key table

This table explains the main symbols used in the equations. When studying, cover the meaning column and check whether you can explain each symbol clearly.

Symbol	Meaning	Plain-English reading
x	Text prompt/instruction	User text or context.
v	Visual input	Image tokens/features.
y	Generated answer	Output token sequence.
d	Retrieved document	External evidence used by RAG.
m_t	Loss mask	Whether token t contributes to SFT loss.
W	Frozen weight matrix	Original pretrained parameter.
B, A	Low-rank adapter matrices	Trainable LoRA factors.
yw, y_l	Winner/loser responses	Preferred and rejected outputs.

Symbol	Meaning	Plain-English reading
π_{ref}	Reference policy	Frozen baseline model in DPO.
r_i	Reward	Score for completion i .

7. Main code prompt / implementation

skeleton This is not meant to replace the lecture code. It is a compact prompt showing the computational pattern you should recognize in homework and exams.

LoRA/SFT adaptation prompt

```
base_model.freeze() attach_lora_adapters(base_model, target_modules=['q_proj','v_proj','o_proj'])
for batch in instruction_data:
    logits = model(batch.text, batch.image)
    loss = masked_cross_entropy(logits, batch.labels, batch.assistant_mask)
    loss.backward()
    optimizer.step()
```

How to explain the code in words

Start by identifying the input, the model output, the objective being minimized, and the update rule. Then connect the code back to the formulas and the method table.

8. Common exam traps

Trap	Correct way to think
Calling few-shot parameter	learning Few-shot prompting changes context, not weights.
Using SFT for missing evidence	RAG is often cleaner when knowledge is missing.
Thinking LoRA is an objective	LoRA is a parameterization; SFT is an objective.

9. Exam-style sample questions with

answers These are written in the style of conceptual exam answers. Use them to practice explaining, not only recognizing terms.

Question 1

Why is the adaptation ladder not a simple weak-to-strong ranking?

Sample answer

Methods change different things: prompt, context, evidence, weights, preferences, or rewards.

Question 2

Why adapt attention and MLP projections?

Sample answer

Attention changes routing; MLP changes transformation. Many tasks need both.

Question 3

How is VLM generation mapped to RL?

Sample answer

State is prompt/image/prefix, action is next token, trajectory is full answer, reward scores the answer.

Question 4

Why use RAG before SFT for missing knowledge?

Sample answer

RAG supplies evidence; SFT imitates outputs but may not supply current facts.

Question 5

How do RAG, LoRA SFT, and RL operate at different layers?

Sample answer

RAG changes evidence, LoRA changes trainable parameters, RL changes reward-optimized policy behavior.

10. Flashcards for rapid review

Use these for active recall. Cover the answer column and try to answer before checking.

Card 1

Q: What changes in zero-shot?

A: Prompt only.

Card 2

Q: Does few-shot update weights?

A: No.

Card 3

Q: What does RAG add?

A: Retrieved evidence.

Card 4

Q: What does SFT optimize?

A: Next-token likelihood on labels.

Card 5

Q: What is LoRA?

A: Low-rank trainable update.

Card 6

Q: What is QLoRA?

A: Quantized base plus LoRA adapters.

Card 7

Q: What does DPO use?

A: Preference pairs.

Card 8

Q: What is RLOO?

A: Leave-one-out reward baseline.

Card 9

Q: What must you remember about Common VLM policy?

A: $p_{\theta}(y|x_{\text{text}}, x_{\text{image}})$ autoregressively generates responses.

Card 10

Q: What must you remember about Zero/few-shot?

A: No parameter update; behavior changes through context.

Card 11

Q: What must you remember about RAG?

A: Inject retrieved evidence at inference time.

Card 12

Q: What must you remember about SFT?

A: Train on labeled examples with next-token likelihood.

Card 13

Q: What must you remember about LoRA/QLoRA?

A: Freeze base weights and learn low-rank updates.

Card 14

Q: What must you remember about DPO/RL?

A: Use preferences or rewards to align outputs.

Card 15

Q: What does this formula represent:

$p_{\theta}(y|x, v) = \prod_t p_{\theta}(y_t|y_{<t}, x, v)$?

A: Autoregressive VLM policy.

Card 16

Q: What does this formula represent:

$p(y|x, v) = \sum_d p(y|x, v, d)p(d|x, v)$?

A: RAG evidence marginalization.

Card 17

Q: What does this formula represent: $L = -\sum_t m_t \log p_{\theta}(y_t|y_{<t}, x, v)$?

A: Masked supervised fine-tuning.

Card 18

Q: What does this formula represent: $W' = W + BA$?

A: LoRA low-rank update.

Card 19

Q: What does this formula represent: $LDPO = -\log \sigma(\beta[(\log \pi_{\theta}(y_w) - \log \pi_{\theta}(y_l)) - (\log \pi_{\text{ref}}(y_w) - \log \pi_{\text{ref}}(y_l))])$?

A: Preference alignment.

Card 20

Q: What does this formula represent: $A_i = r_i - \text{mean}_{\{j \neq i\}} r_j$?

A: RLOO relative reward baseline.

Card 21

Q: What does x mean?

A: Text prompt/instruction. User text or context.

Card 22

Q: What does v mean?

A: Visual input. Image tokens/features.

Card 23

Q: What does y mean?

A: Generated answer. Output token sequence.

Card 24

Q: What does d mean?

A: Retrieved document. External evidence used by RAG.

Card 25

Q: What does m_t mean?

A: Loss mask. Whether token t contributes to SFT loss.

Card 26

Q: What does W mean?

A: Frozen weight matrix. Original pretrained parameter.

Card 27

Q: What does B, A mean?

A: Low-rank adapter matrices. Trainable LoRA factors.

Card 28

Q: What does y_w, y_l mean?

A: Winner/loser responses. Preferred and rejected outputs.

Card 29

Q: What does π_{ref} mean?

A: Reference policy. Frozen baseline model in DPO.

Card 30

Q: What does r_i mean?

A: Reward. Score for completion i .

Card 31

Q: What is the key idea of Adaptation ladder?

A: Methods differ by what they change.

Prompting changes instructions. Few-shot adds demonstrations. RAG adds external evidence.

SFT changes weights using labels. DPO and RL align behavior using preferences or

Card 32

Q: What is the key idea of Zero-shot and few-shot?

A: No gradient update occurs. The model behavior changes because the Transformer conditions on the full context, including

Card 33

Q: What is the key idea of RAG?

A: Retrieval-augmented generation is best when the model lacks knowledge. It conditions the answer on retrieved evidence rather than relying only on internal

Card 34

Q: What is the key idea of SFT and masking?

A: Supervised fine-tuning trains the model on labeled responses. In chat data, the loss usually applies only to assistant tokens, not user/system

Card 35

Q: What is the key idea of LoRA and QLoRA?

A: LoRA freezes the base weight and learns a low-rank update. QLoRA reduces memory further by quantizing base weights while training low-rank

Card 36

Q: What is the key idea of DPO and RL?

A: DPO compares preferred and rejected responses relative to a reference policy. RL treats token generation as a sequence of actions and optimizes expected

Card 37

Q: What happens in the step: Start with prompting?

A: Change instruction without changing parameters.

Card 38

Q: What happens in the step: Add evidence?

A: Use RAG when failure is missing or stale knowledge.

Card 39

Q: What happens in the step: Fine-tune efficiently?

A: Use SFT/LoRA for persistent task behavior.

Card 40

Q: What happens in the step: Align behavior?

A: Use DPO/RL when preferences or rewards matter.

Card 41

Q: Can you choose adaptation by failure mode?

A: Missing facts->RAG; task behavior->SFT/LoRA; preference->DPO/RL.

Card 42

Q: Can you explain LoRA vs SFT?

A: SFT objective; LoRA parameterization.

Card 43

Q: Can you explain masking?

A: Only desired answer tokens train.

Card 44

Q: Can you explain DPO?

A: Relative preference against reference policy.

Card 45

Q: Why is the adaptation ladder not a simple weak-to-strong ranking?

A: Methods change different things: prompt, context, evidence, weights, preferences, or rewards.

Card 46

Q: Why adapt attention and MLP projections?

A: Attention changes routing; MLP changes transformation. Many tasks need both.

Card 47

Q: How is VLM generation mapped to RL?

A: State is prompt/image/prefix, action is next token, trajectory is full answer, reward scores the answer.

Card 48

Q: Why use RAG before SFT for missing knowledge?

A: RAG supplies evidence; SFT imitates outputs but may not supply current facts.

Card 49

Q: How do RAG, LoRA SFT, and RL operate at different layers?

A: RAG changes evidence, LoRA changes trainable parameters, RL changes reward-optimized policy behavior.

Card 50

Q: In plain English, why is $p_{\theta}(y|x,v) = \prod_t p_{\theta}(y_t|y_{<t},x,v)$ important?

A: It captures: Autoregressive VLM policy. Use it as a compact exam sentence, then define each symbol.

Card 51

Q: In plain English, why is $p(y|x,v) = \sum_d p(y|x,v,d)p(d|x,v)$ important?

A: It captures: RAG evidence marginalization. Use it as a compact exam sentence, then define each symbol.

Card 52

Q: In plain English, why is $L = -\sum_t m_t \log p_{\theta}(y_t|y_{<t},x,v)$ important?

A: It captures: Masked supervised fine-tuning.

Use it as a compact exam sentence, then define each symbol.

Card 53

Q: In plain English, why is $W' = W + BA$ important?

A: It captures: LoRA low-rank update. Use it as a compact exam sentence, then define each symbol.

Card 54

Q: In plain English, why is $LDPO = -\log \sigma(\beta[(\log \pi_{\theta}(y_w) - \log \pi_{\theta}(y_l)) - (\log \pi_{\text{ref}}(y_w) - \log \pi_{\text{ref}}(y_l))])$ important?

A: It captures: Preference alignment. Use it as a compact exam sentence, then define each symbol.

Card 55

Q: In plain English, why is $A_i = r_i - \text{mean}_{\{j \neq i\}} r_j$ important?

A: It captures: RLOO relative reward baseline.

Use it as a compact exam sentence, then define each symbol.

Explanation

Chapter 9: Training and Adapting Vision-Language Models

How this section is organized

Each item below is a heading taken from the corresponding Jupyter notebook in the same order it appears there. The explanation beside it tells you what that notebook section is trying to teach and why it matters for the review.

Source notebook

Chapter_9_Training_and_Adapting_VLMs(1).ipynb

Headings included

88

Notebook headings with explanation

1. Chapter 9 - Training and Adapting Vision-Language Models

This notebook heading introduces chapter 9 - training and adapting vision-language models and explains how it supports the chapter concepts, equations, method steps, or implementation workflow.

2. Learning Outcomes

By the end of this chapter, students should be able to: 1. Explain the adaptation ladder for vision-language models (VLMs) from zero-shot and few-shot prompting to RAG, supervised fine-tuning, parameter-efficient fine-tuning, preference optimization, and RL-based adaptation.

3. Overview

The chapter is organized according to the methods below. The opening sections load the dataset, choose a working subset, and define a common evaluation format so that the later method-by-method comparisons remain consistent.

4. Method table and budget ladder

This table defines the numbering used throughout the notebook. Method; Method; Changes model weights?; Main cost; Typical use 1; Zero-shot; No; almost free; get a baseline 2; Few-shot; No; low; format, style, and task steering 3; RAG; No; low to medium; inject external knowledge 4; SFT; Yes; medium; teach task behavior directly 5; LoRA / QLoRA; Yes, but only adapters; medium; practical specialization 6; DPO; Yes; medium to high; improve preference alignment 7; RLHF / PPO; Yes; highest; full online.

5. Load the real Rare Species dataset

We load the real dataset from Hugging Face Datasets. The goal is not to use the entire dataset for the lab. The notebook also includes 2 nearby code cells for implementation, demonstration, or visualization of this heading.

6. Choose a biologically meaningful working subset

The original dataset is much larger than what we need for repeated experiments in one notebook. We therefore construct a subset that is: - taxonomically coherent, - large enough to be nontrivial, - small enough to support repeated inference and tuning runs. The notebook also includes 4 nearby code cells for implementation, demonstration, or visualization of this heading.

7. Load the Qwen VLM

We use Qwen/Qwen2.5-VL-3B-Instruct throughout. A 4-bit load makes it practical to keep a strong base model in memory while reserving room for adapters, optimizer state, and batches on a single high-end GPU. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.

8. Shared utility functions

The next cell defines reusable utilities for: 1. creating a consistent task prompt, 2. calling the model, 3. parsing structured outputs, 4. evaluating predictions under the same format. Keeping the task definition and output format fixed is important because later comparisons should reflect the method itself rather than a hidden change in prompt wording or evaluation logic. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.

9. Mathematical setup for the common VLM policy	Before comparing methods, write the model once in a form that all later sections can reuse. A modern autoregressive vision-language model defines a conditional distribution $p_{\theta}(y \mid x, v) = \prod_{t=1}^T p_{\theta}(y_t \mid y_{<t}, x, v)$.
10. What changes across methods?	Methods 1 and 2: keep θ fixed and modify the context. - Method 3: keep θ fixed and augment the context with retrieved evidence. - Methods 4 through 7: update some or all trainable parameters. Throughout this lecture, the rare-species task uses the same basic input-output structure.
11. Method — Zero-shot prompting	This notebook heading introduces method — zero-shot prompting and explains how it supports the chapter concepts, equations, method steps, or implementation workflow.
12. Idea	In zero-shot prompting we do not change the model parameters and we do not add retrieved external knowledge. We present the pretrained VLM with an image and an instruction and ask it to solve the task directly.
13. Mathematical view	Let the text instruction be x , let the image be v , and let the generated answer be $y = (y_1, \dots, y_T)$. The VLM defines $p_{\theta}(y \mid x, v) = \prod_{t=1}^T p_{\theta}(y_t \mid y_{<t}, x, v)$.
14. Why does it work?	Zero-shot works because the pretrained VLM has already learned broad visual and textual regularities during large-scale pretraining and instruction tuning. If the current task is close to that prior experience, a well-written prompt may be enough.
15. Code Summary	The next cells do three things: 1. build a single multimodal prompt for one rare-species example, 2. run the Qwen VLM without any weight updates, 3. evaluate the same prompt pattern over a small held-out subset. That gives us a clean baseline before we move to stronger but more expensive methods. The notebook also includes 2 nearby code cells for implementation, demonstration, or visualization of this heading.
16. Issues with Zero-Shot	Prompt sensitivity Because the parameters are fixed, the result can change noticeably when we alter wording, output format, or the amount of label guidance in the prompt. No new knowledge If the model does not already know the relevant species distinctions, zero-shot prompting cannot inject them.
17. Method - Few-shot / in-context learning	This notebook heading introduces method - few-shot / in-context learning and explains how it supports the chapter concepts, equations, method steps, or implementation workflow.
18. Why can few-shot learning work if θ does not change?	This is the central conceptual question. The short answer is: the context changes even though the weights do not.
19. Why does it potentially work?	Few-shot prompting can work for at least four reasons. Format induction If every example uses the same output schema, the model infers that the new answer should follow that same schema.
20. What few-shot learning is <i>*not*</i>	Few-shot prompting is not: - gradient-based learning on the new examples, - permanent storage of those examples in the weights, - guaranteed generalization beyond the current context window. Once the prompt disappears, the adaptation disappears.
21. Why can it fail?	Few-shot prompting can fail because: - the demonstrations are poorly chosen, - the examples are too long and crowd the context window, - the model matches superficial patterns rather than the intended task, - or the task requires information the model never learned during pretraining.

22. Issues	Context is expensive: every extra example consumes tokens and, for VLMs, image processing budget. - Demonstration order matters. - A demonstration can help one class but hurt another if it is visually confusing. - Unlike SFT, few-shot prompting does not create a reusable adapted model.
23. References for Few Shot learning	B., Mann, B., Ryder, N., et al. *Language Models are Few-Shot Learners* - Min, S., Lyu, X., Holtzman, A., et al. *Rethinking the Role of Demonstrations: What Makes In-Context Learning Work?* - Xie, S. M., Raghunathan, A., Liang, P., Ma, T. *An Explanation of In-context Learning as Implicit Bayesian Inference* - Lv, S.-L., Chen, Y.-Y., Zhou, Z., Yang, M., Guo, L.-Z. *BMIP: Bi-directional Modality Interaction Prompt Learning for VLM* The notebook also includes 2 nearby code cells for implementation, demonstration, or visualization of this heading.
24. Issues With Few-Shot Learning	Context-window cost Each demonstration consumes tokens, and multimodal examples are expensive in both memory and latency. Example selection matters Poor or unrepresentative demonstrations can hurt performance.
25. Method - Retrieval-Augmented Generation (RAG)	This notebook heading introduces method - retrieval-augmented generation (rag) and explains how it supports the chapter concepts, equations, method steps, or implementation workflow.
26. Simple RAG	This notebook heading introduces simple rag and explains how it supports the chapter concepts, equations, method steps, or implementation workflow.
27. RAG with Backpropagation	This notebook heading introduces rag with backpropagation and explains how it supports the chapter concepts, equations, method steps, or implementation workflow.
28. Basic probabilistic picture	Let: - x be the text instruction, - v be the image, - $q(x, v)$ be the query representation produced from the instruction and image, - z be one retrieved document, - Z be the document collection, - η be the retriever parameters, - θ be the generator parameters, - $y = (y_1, \dots, y_T)$ be the output sequence. A standard latent-document RAG model writes $p_{\theta, \eta}(y \mid x, v) = \sum_z \text{in } Z p_{\eta}(z \mid q(x, v)), p_{\theta}(y \mid x, v, z)$.
29. What is the gold target in RAG?	At training time, the gold target is usually the desired answer sequence $y^* = (y_1^*, \dots, y_T^*)$. For our notebook, y^* is the structured rare-species answer we want the model to produce, such as the correct scientific name, family, genus, and a short evidence sentence.
30. Marginal log-likelihood and why the equations jump	The quantity actually optimized in a differentiable RAG formulation is often the log marginal likelihood of the gold answer: $L_{\text{RAG}}(\theta, \eta) = -\log p_{\theta, \eta}(y^* \mid x, v)$. Using the latent-document expansion, this becomes $L_{\text{RAG}}(\theta, \eta) = -\log \left[\sum_z \text{in } Z p_{\eta}(z \mid q(x, v)), p_{\theta}(y^* \mid x, v, z) \right]$.
31. Posterior responsibility weights $w(z)$	A very useful quantity is $w(z) = \frac{p_{\eta}(z \mid q(x, v))}{\sum_{z' \text{ in } Z} p_{\eta}(z' \mid q(x, v))}$, $p_{\theta}(y^* \mid x, v, z)$. This is a posterior responsibility weight.
32. Gradient with respect to the generator	Start from $\log p_{\theta, \eta}(y^* \mid x, v) = \log \left[\sum_z p_{\eta}(z \mid q(x, v)), p_{\theta}(y^* \mid x, v, z) \right]$. Differentiate with respect to θ : $\nabla_{\theta} \log p_{\theta, \eta}(y^* \mid x, v) = \sum_z w(z), \nabla_{\theta} \log p_{\theta}(y^* \mid x, v, z)$.
33. Gradient with respect to the retriever	Similarly, the retriever gradient is $\nabla_{\eta} \log p_{\theta, \eta}(y^* \mid x, v) = \sum_z w(z), \nabla_{\eta} \log p_{\eta}(z \mid q(x, v))$. So the retriever is pushed to assign more mass to documents that, in hindsight, explain the gold answer well.
34. Why this matters even if we use a frozen retriever	In our example implementation, we use a lightweight TF-IDF retriever and do not backpropagate through retrieval. That is common in real systems because it is simpler and cheaper.

35. Token-level intuition	Another way to understand RAG is at the token level. Without retrieval, the next-token distribution is $p_{\theta}(y_t \mid y_{<t}, x, v)$.
36. Issues with RAG	Retrieval quality is a bottleneck If the correct species card is not retrieved, the generator cannot use it. Retrieval errors can be amplified A confident but wrong retrieved document can steer the model toward a persuasive incorrect answer.
37. Method — Supervised fine-tuning (SFT)	This notebook heading introduces method — supervised fine-tuning (sft) and explains how it supports the chapter concepts, equations, method steps, or implementation workflow.
38. Intuition behind the loss	At each position, the model predicts a distribution over the next token. The cross-entropy loss simply says: > the correct token should receive higher probability than it currently does.
39. Can SFT be related to a score?	Yes, but we must be careful about what "score" means. In diffusion models, "score" usually means $\nabla_x \log p(x)$, which is a gradient with respect to the input.
40. What is an assistant token?	In chat-style training, the tokenized conversation contains: - system-role text, prompt system component - user-role text, prompt role component - assistant-role text, The reply from the system - plus special control markers used by the chat template. When practitioners say assistant tokens, they usually do not mean one special vocabulary token.
41. Important practical note for Qwen2.5-VL	The <code>assistant_only_loss=True</code> path in TRL depends on the tokenizer or processor being able to return an assistant-token mask. Hugging Face documents that this mask is only available for chat templates that support % generation % blocks.
42. Data structure and label masking	The code uses a conversational representation because the processor knows how to place the image markers correctly for the model. During training, the causal objective uses a one-token shift, and padding positions are ignored.
43. References for SFT	Ouyang, L., Wu, J., Jiang, X., et al. *Training Language Models to Follow Instructions with Human Feedback* - Hugging Face TRL documentation. *SFT Trainer* - Hugging Face Transformers documentation. *Tokenizer.apply_chat_template and assistant token masks* - Qwen2.5-VL community discussion. *Chat template of AutoTokenizer does not work on assistant mask tokens*
44. Build the supervised dataset	The code below converts our rare-species examples into a standard message format suitable for instruction-style multimodal training. This is conceptually important because a lot of modern post-training work is really about dataset formatting. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
45. Data structure, assistant spans, and label masking	For each example, we build a conversation with: 1. a system message, 2. a user message containing the image and instruction, 3. an assistant message containing the desired structured answer. From the model's point of view, only the assistant response should be predicted.
46. Issues with SFT	Label quality dominates performance If the target answers are noisy, inconsistent, or partially wrong, the model will learn those inconsistencies. Mode collapse toward one answer style SFT encourages imitation of the provided targets.
47. Approach — LoRA / QLoRA	This notebook heading introduces approach — lora / qlora and explains how it supports the chapter concepts, equations, method steps, or implementation workflow.
48. Base layer mathematics	Suppose a linear layer uses weight matrix W in $R^{d_{out}} \times d_{in}$ and applies it to an input vector x in $R^{d_{in}}$. Its output is $h = Wx$, where h in $R^{d_{out}}$.

49. Why a low-rank update can work	A downstream task rarely needs to rewrite every degree of freedom in every large matrix. Many useful task adaptations can be represented as a structured directional correction to the original weight.
50. Which layers are actually adapted in a transformer VLM?	In this notebook we target the following linear sublayers: - q_proj - k_proj - v_proj - o_proj - gate_proj - up_proj - down_proj These correspond to two major subsystems. Attention projections For one attention block, let: - X in $R^{L \times d}$ be the input hidden states, - L be the sequence length, - d be the model width.
51. What changes in QLoRA?	QLoRA leaves the LoRA idea intact but stores the frozen base model in quantized form, typically 4-bit, while the LoRA adapters remain trainable in higher precision. So conceptually: - the base model stays frozen and compressed, - the optimizer only updates the adapter weights, - backpropagation still flows through the forward pass, but only adapter parameters receive updates.
52. The loss optimized in LoRA / QLoRA	LoRA does not invent a new task loss. For supervised tuning it still optimizes $L_{SFT}(\theta_{base}, A, B) = - \sum_{i,t} m_{i,t} \log p(y_{i,t} x_{i,t}^{*}, x_i, v_i)$.
53. Intuition behind the LoRA loss	The loss is still telling the model to increase the probability of the correct next token. The only restriction is *where the model is allowed to change*.
54. Inspecting target modules	The next code cell verifies that suffixes such as q_proj, v_proj, and down_proj really occur inside the loaded model. This is important because LoRA acts on concrete named modules, not on abstract equations. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
55. Evaluation and interpretation	The evaluation stage uses the same species-identification task as the earlier methods. That keeps the comparison fair: we are changing the adaptation method, not secretly changing the task. The notebook also includes 2 nearby code cells for implementation, demonstration, or visualization of this heading.
56. Issues with LoRA	Rank r is a real modeling choice If r is too small, the adapter may not have enough capacity. If it is too large, some of the memory and regularization benefits disappear.
57. Background — Reinforcement learning Primer	This notebook heading introduces background — reinforcement learning primer and explains how it supports the chapter concepts, equations, method steps, or implementation workflow.
58. Basic objects	An episodic reinforcement-learning problem uses: - s_t : state at time step t , - a_t : action at time step t , - r_t : reward observed after that action, - $\pi_{\theta}(a_t s_t)$: the policy distribution. For sequence generation: - the state s_t is the prompt, image, and generated prefix available at step t , - the action a_t is the next generated token, - the policy π_{θ} is the language model or vision-language model.
59. Return and objective	With discount factor γ in $[0,1]$, the return from time t is $G_t = \sum_{k=t}^T \gamma^{k-t} r_k$. Here: - G_t is the total discounted future reward starting at time t , - γ controls how much future rewards are discounted, - T is the terminal step.
60. Policy-gradient intuition	A classic policy-gradient identity is $\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t s_t) G_t \right]$. Interpretation: - if an action was followed by high return, increase its probability, - if an action was followed by poor return, decrease its probability.
61. Why PPO or PPO-like methods enter RLHF	Raw policy gradients can be unstable. PPO stabilizes learning by discouraging very large policy changes in one update.

62. How this relates to a VLM	In a VLM: - the state s_t contains the image, the textual prompt, and the previously generated tokens, - the action a_t is the next token, - the reward may depend on correctness, faithfulness, format, calibration, or human preference, - the rollout is a generated answer sequence. So RLHF for a VLM is just token-level policy optimization over multimodal prompts.
63. Important practical note	For VLMs, the *cleanest officially documented online RL path today* is often not classical PPO but trainers such as RLOO or Online DPO. We still study PPO because it gives the conceptual template for KL-regularized policy optimization.
64. Mapping the RL notation back to a VLM	For language or vision-language generation, the RL objects become: - state s_t : the prompt, image, and generated prefix so far, - action a_t : the next generated token, - policy $\pi_\theta(a_t \mid s_t)$: the VLM next-token distribution, - trajectory τ : the full generated response, - reward: a scalar quality score for the completed answer. This bridge is the key to understanding RLHF.
65. Method — Preference tuning with DPO	This notebook heading introduces method — preference tuning with dpo and explains how it supports the chapter concepts, equations, method steps, or implementation workflow.
66. Preference data	For each prompt-image pair (x, v) we construct: - a chosen response y_w , - a rejected response y_l . Here: - x is the prompt, - v is the image, - y_w is the preferred answer, - y_l is the rejected answer, - π_θ is the trainable policy, - π_{ref} is the frozen reference policy, - β controls the sharpness of the preference update.
67. DPO loss	A standard DPO loss is $L_{DPO}(\theta) = -\log \sigma(\beta [\log \pi_\theta(y_w \mid x, v) - \log \pi_\theta(y_l \mid x, v) - \log \pi_{ref}(y_w \mid x, v) + \log \pi_{ref}(y_l \mid x, v)])$. If we define the preference margin $m = \beta [\log \pi_\theta(y_w \mid x, v) - \log \pi_\theta(y_l \mid x, v) - \log \pi_{ref}(y_w \mid x, v) + \log \pi_{ref}(y_l \mid x, v)]$, then $L_{DPO}(\theta) = -\log \sigma(m)$.
68. Why the sigmoid appears	The sigmoid converts the margin into a smooth binary-comparison probability. The resulting loss behaves like logistic regression on pairwise preferences.
69. Why is DPO often easier than full RLHF?	DPO avoids: - explicit reward-model fitting during the policy update, - sampling rollouts during training, - advantage estimation, - PPO clipping logic. It converts preference learning into a stable supervised-style optimization problem over pairs of outputs.
70. DPO vs. KTO	KTO is another alternative to DPO and is sometimes easier to implement. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
71. Build a preference dataset from the rare-species task	The next code cell constructs preference pairs from the same underlying rare-species examples. A single labeled example can support several views: - an SFT target, - a chosen response, - a rejected response, - multiple preference pairs with different error patterns. The notebook also includes 4 nearby code cells for implementation, demonstration, or visualization of this heading.
72. DPO training code	The next cells define a practical DPO setup using TRL. DPO can be combined with PEFT adapters, so preference optimization does not necessarily require a full-model update. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
73. Method Reinforcement learning from feedback: reward modeling + online RL	This notebook heading introduces method reinforcement learning from feedback: reward modeling + online rl and explains how it supports the chapter concepts, equations, method steps, or implementation workflow.

74. What is fully implemented below?	This notebook now includes two runnable parts: Reward modeling We train a small reward model on the same rare-species preference pairs. This is the classical "fit a scalar judge from pairwise feedback" part of RLHF.
75. Reward-model objective	A reward model $r_{\phi}(x, y)$ or $r_{\phi}(x, v, y)$ maps a prompt and answer to a scalar score. Given a preferred answer y_w and a rejected answer y_l , a standard pairwise loss is $L_{RM}(\phi) = -\log \sigma(r_{\phi}(x, v, y_w) - r_{\phi}(x, v, y_l))$.
76. KL-regularized policy objective	Once a reward signal exists, the policy objective is conceptually $\max_{\theta} \mathbb{E}_{\pi_{\theta}}[\log p_{\theta}(x, v) \log r(x, v, y)] - \beta \text{KL}(\pi_{\theta} \parallel \pi_{\text{ref}})$. Here: π_{θ} is the current policy, π_{ref} is a frozen reference policy, $r(x, v, y)$ is either a learned reward model score or a handcrafted/verifiable reward, β controls how strongly we penalize drift away from the reference model.
77. PPO-style intuition	PPO uses the ratio $r_t(\theta) = \frac{\pi_{\theta}(a_t s_t)}{\pi_{\theta_{\text{old}}}(a_t s_t)}$, and the clipped surrogate $L^{\text{CLIP}}(\theta) = \mathbb{E}[\min(r_t(\theta) A_{\hat{t}}, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) A_{\hat{t}})]$. Here: a_t is the generated token at step t , s_t is the state consisting of image, prompt, and generated prefix, θ_{old} are the parameters that generated the rollout, $A_{\hat{t}}$ is the advantage estimate, ϵ is the clipping radius.
78. Why we are using RLOO for the runnable VLM stage	TRL's documentation explicitly supports VLM training with RLOO and lists Qwen/Qwen2.5-VL-3B-Instruct among the tested models. That makes RLOO the most direct fully runnable online RL option for the same VLM used throughout this notebook.
79. Code walkthrough	The next cells do the following: 1. build a text-only reward-model dataset from the rare-species preference pairs, 2. train a small reward model with RewardTrainer, 3. build a conversational VLM RL dataset, 4. define reward functions for format, taxonomic correctness, and calibration, 5. run RLOO on the same Qwen VLM, 6. reload the adapted policy and evaluate it.
80. RLOO: idea and core math	RLOO stands for REINFORCE Leave-One-Out. It is an RL-based alignment method in which the model generates several candidate completions for the same prompt, each completion receives a reward, and the policy is updated to increase the probability of completions that score better than the others. The notebook also includes 8 nearby code cells for implementation, demonstration, or visualization of this heading.
81. Revisit the method table: empirical and conceptual comparison	This section returns to the original method table and summarizes the methods on the same task. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
82. Method-by-method comparison	Method — Zero-shot - cheapest, - fastest, - best first baseline, - limited specialization. Method — Few-shot - still no training, - good for formatting and task steering, - limited by context length and exemplar quality.
83. Fine-tuning case study summary table	Case study: Qwen2.5-VL-3B-Instruct on Rare Species, using reduced image resolution through Qwen's min_pixels / max_pixels controls. These are rough engineering estimates, not official benchmarks.
84. Core methods	J., Shen, Y., Wallis, P., et al. *LoRA: Low-Rank Adaptation of Large Language Models* 2. QLoRA Detmiers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L. *QLoRA: Efficient Finetuning of Quantized LLMs* 3.
85. Dataset and biodiversity context	Rare Species dataset card *imageomics/rare-species* 14. BioCLIP Stevens, S., Wu, J., Thompson, M.
86. Practical tooling	Qwen2.5-VL model card *Qwen/Qwen2.5-VL-3B-Instruct* 16. Transformers tokenizer documentation *apply_chat_template and assistant-token masks* 17.

87. Recent 2024-2026 papers from major venues

ICLR 2025 — theory for token-level RAG Xu, S., Pang, L., Shen, H., Cheng, X. *A Theory for Token-Level Harmonization in Retrieval-Augmented Generation* 26.
ICLR 2025 — hybrid / faithful RAG Ma, S., Xu, C., Jiang, X., et al. *Think-on-Graph 2.0: Deep and Faithful Large Language Model Reasoning with Knowledge-guided Retrieval-Augmented Generation* 27.

88. Note on IJCAI 2026

As of the date of this notebook, the IJCAI-ECAI 2026 conference website is public, but the 2026 proceedings are not yet public. That is why the newest IJCAI papers cited here are from 2025. - IJCAI-ECAI 2026 conference site